Secure Rejection Demo

Showing how the secure RSA/JWT app rejects invalid signatures and tokens

2026-05-30

Overview

This supplement demonstrates the defensive side of the RSA/JWT lab. The goal is to show that the secure app does not only accept valid cryptographic material; it also rejects invalid signatures, tampered tokens, unsigned tokens, and malformed tokens with clear logs.

How To Run

Start the secure app and dashboard:

```
docker compose -f docker-compose.secure.yml up --build secure-app dashboard
```

Run the rejection demo:

```
docker compose -f docker-compose.secure.yml --profile demo run --rm secure-rejection-demo
```

Open the dashboard:

```
http://localhost:8090
```

What The Demo Sends

The `secure-rejection-demo` client sends five negative test cases.

1. Wrong Payload With A Real Signature

The client asks the secure app to sign:

```
transfer=100&to=alice
```

Then it verifies that same signature against:

```
transfer=9000&to=mallory
```

Expected result:

```
HTTP 401
{"valid":false}
```

Dashboard event:

```
secure_reject_wrong_payload_signature
```

This proves the signature is bound to the exact payload.

2. Corrupted Signature

The client flips the last Base64 character of a real signature and sends it to `/secure/verify`.

Expected result:

```
HTTP 401 or HTTP 400
```

Dashboard event:

```
secure_reject_corrupt_signature
```

If the corrupted bytes still decode as Base64 but fail PSS verification, the app returns `401`. If the corruption makes the signature structurally invalid, the app can return `400`.

3. Tampered JWT Payload

The client asks `/token/issue` for a valid user token, then replaces the payload with an admin claim while keeping the original signature.

The signing input changes from:

```
base64url(header).base64url(original_payload)
```

to:

```
base64url(header).base64url(tampered_admin_payload)
```

The original signature no longer matches.

Expected result:

```
HTTP 401
{"valid":false,"payload":""}
```

Dashboard events:

```
secure_reject_tampered_token_payload
token_verify_reject reason=signature_mismatch
```

4. Unsigned `alg=none` JWT

The client sends an unsigned admin token:

```
{"alg":"none","typ":"JWT"}
```

with payload:

```
{"sub":"mallory","role":"admin"}
```

The secure app rejects it because the verifier pins `PS256` server-side.

Expected result:

```
HTTP 401
{"valid":false,"reason":"alg_not_allowed"}
```

Dashboard events:

```
secure_reject_unsigned_alg_none_token
token_verify_reject reason=alg_not_pinned
```

5. Malformed Token

The client sends:

```
not-a-jwt
```

A JWT must have three dot-separated parts. This input does not.

Expected result:

```
HTTP 400
{"error":"bad_token_shape"}
```

Dashboard events:

```
secure_reject_malformed_token
token_verify_reject reason=bad_token_shape
```

Dashboard Search Terms

```
secure_reject_
token_verify_reject
signature_mismatch
alg_not_pinned
bad_token_shape
```

Security Properties Verified

- changed payloads must fail signature verification
- modified JWT claims must invalidate the token
- unsigned tokens must be rejected
- malformed tokens must not be parsed leniently
- logs must show clear rejection reasons without leaking private-key material